Optimized kernels can still contain avoidable work

A stable performance baseline can still contain redundant computation or inefficient target code.

Scalar / vector expressions

Clamp and predicate chains may admit a smaller equivalent expression.

Memory and layout

Adjacent loads or conversion sequences may admit a more efficient representation.

Execution mapping

Reduction structure and vectorization choices can change target utilization.

For each case, establish a legal alternative and compare the resulting artifacts under the same target settings.

[REWORK] How optimization opportunities reach engineers today

Current mechanism	Trigger and engineering work
Nightly performance regression testing	Detect a deterioration against the baseline; investigate or revert the offending change.
Performance issue tickets	Investigate slow cases discovered in already merged code.
Kernel-specific tasks	Inspect generated code and profiling data to make a specified kernel faster.
Optimization epics	Improve an optimization class; task selection and prioritization need clarification.

No proactive search for missed optimizations is currently performed in the project. (To my knowledge)

Proactive discovery currently depends on engineer investigation



Representative repeated investigation activities

Work	Material an engineer needs
Understand the region	Definitions, uses, effects, aliasing, shape/layout and numeric constraints.
Test an alternative	Compiler entry point, enclosing module, target settings, harness and inputs.
Assess the result	Comparable output artifacts, selected metric, conditions and reproduction steps.

Build a framework for reproducible optimization discovery

Develop a framework that searches for beneficial MLIR rewrites and returns a reproducible original / candidate comparison for AscendNPU-IR.



Offload variant construction, repeated checks and reproduction packaging; measure whether the resulting findings justify preparation and review effort.

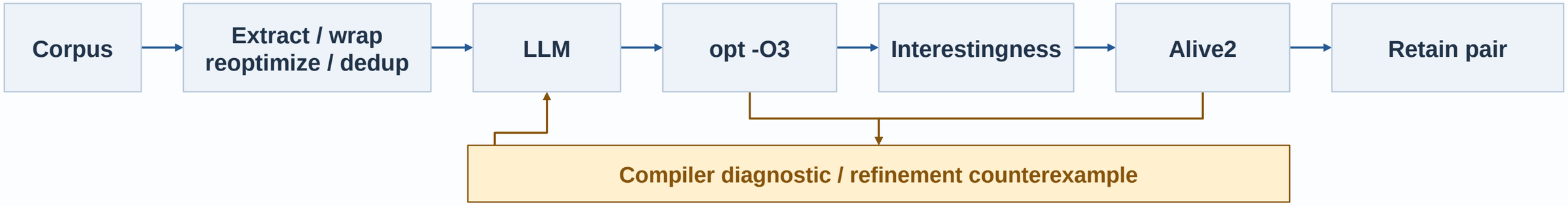
Research directions and planned comparisons

Direction	Baseline or required work	Comparison / question	Evidence to retain
Metrics · Q06	Select one meaningful evaluator.	Static indicator, target model or runtime measurement?	Coverage, later-pass survival and collection cost.
Routes · Q08	Exercise one compile / check route.	Which checkpoints, stops and validation modes compose?	Compatibility matrix and reproducible probes.
Context · Q09	Versioned dialect pack + supplied IR.	Direct context vs on-demand docs or added analyses.	Useful findings, invalid proposals and retrieval cost.
Tools · Q10	Ordinary reading / file editing.	IR-aware editor, restricted rewrites or check tools.	Validity, expressible transformations and total cost.
Discovery · Q07	Engineer supplies code or range.	Prompted selection vs static / cost-model selectors.	Region coverage, useful yield and selection cost.
Feedback / policy	Same conversation; first qualifying finding.	History detail, continuation and budget allocation.	Attempt traces, useful yield and expensive checks.

Optional studies use the same cases, model, checking policy and comparable resource budgets.

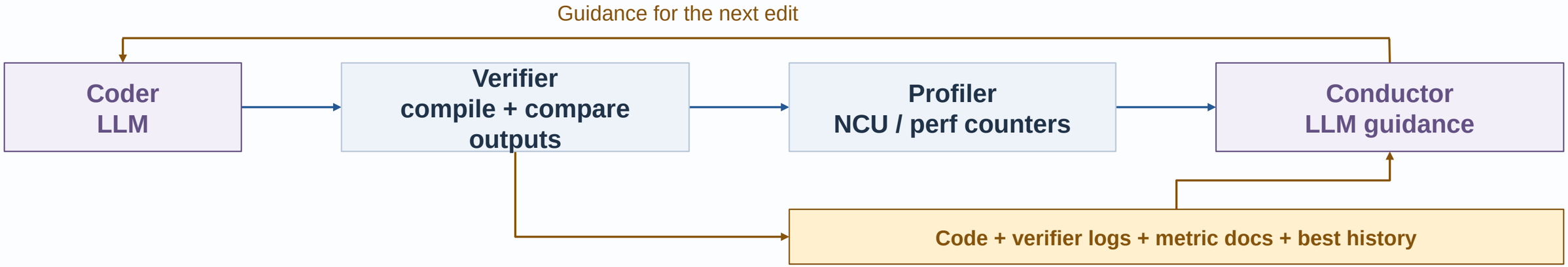
Two paper workflows inform the experiment

LPO — LLVM peephole discovery



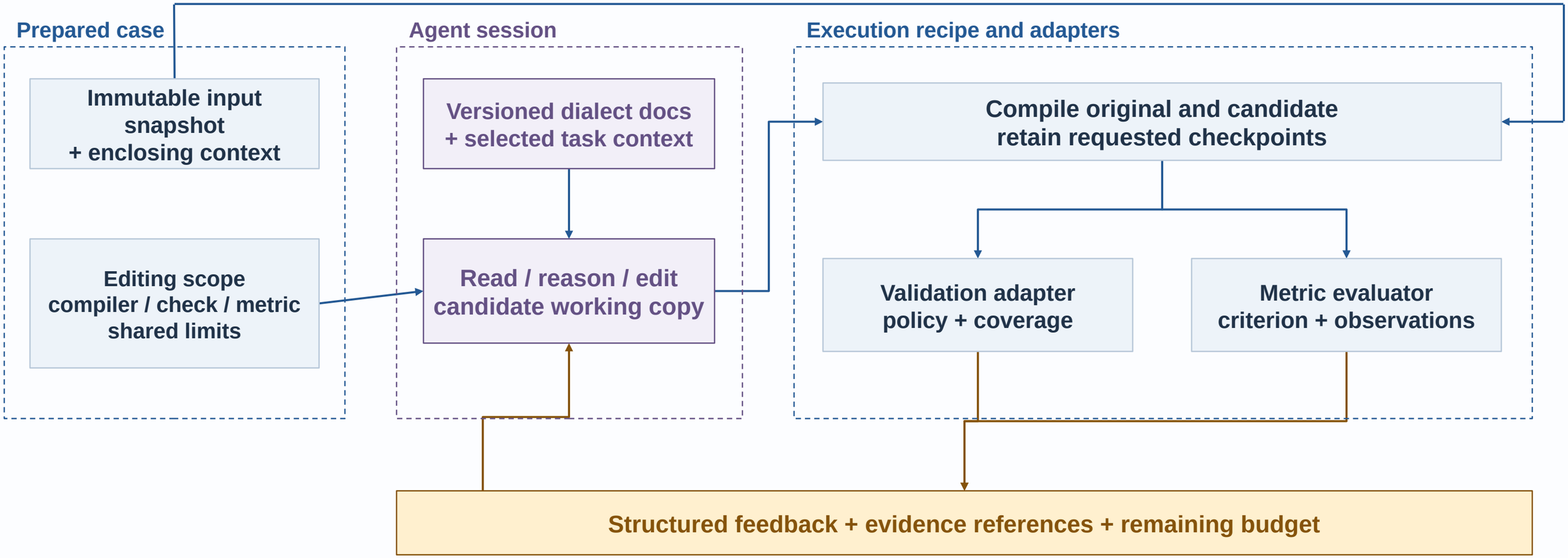
Screen uses instruction count or llvm-mca estimates; syntactic alternatives may also be retained.

PRAGMA — profiling-informed kernel refinement



Editable reconstructions of LPO Fig. 2 and PRAGMA Figs. 1–2. LLVM and CPU/GPU tool assumptions belong to the papers.

Proposed Ascend investigation architecture



Persist snapshots, candidates, operations, feedback delivery state and budget reservations across every stage.

Finding eligibility = required checks pass + selected criterion met + complete runnable bundle.

Candidate transformations and applicability conditions

Family	Candidate change	Conditions / checkpoint information
Arithmetic and clamps	Remove a clamp subsumed by later operations.	Integer width, signedness, overflow flags; exact sequence.
Predicates / selects	Remove a NaN guard before ordered equality.	FP predicate semantics; other uses of the sanitized value.
Memory sequences	Adjacent narrow loads → one wider load.	Bounds, alignment, endianness, memory space and effects.
Vectorization / lowering	Restructure conditions to expose target instructions.	Vector width, legal operations, contraction permissions.
Reduction organization	Change reduction structure or target mapping.	Shape, neutral element, NaN handling, numeric order.
Layout conversions	Eliminate an inverse pair or retain one layout.	Physical layout, aliasing and requirements of consumers.

LPO motivates the first three families. PRAGMA motivates vectorization/reduction. Layout conversion is our additional candidate.

MLIR case — redundant NaN handling

Shared operands %x : f32 %zero = 0.0 : f32 %one = 1.0 : f32

Original

```
%ordinary = arith.cmpf ord, %x, %x : f32
%safe = arith.select %ordinary, %x, %zero : f32
%equal = arith.cmpf oeq, %safe, %one : f32
```

Candidate

```
%equal = arith.cmpf oeq, %x, %one : f32
```

Input class	Original result	Candidate result
NaN	select zero; 0.0 == 1.0 → false	ordered equality → false
Ordinary value	x == 1.0	x == 1.0

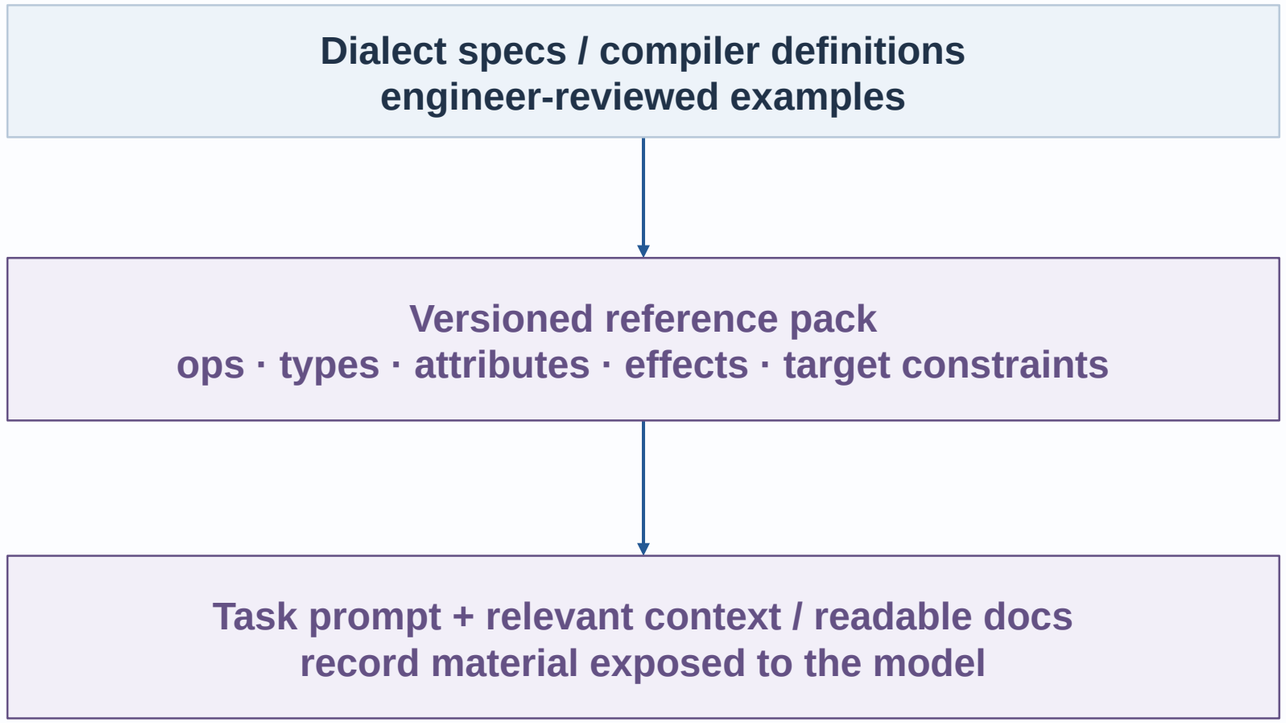
LPO Figure 4(c,f), adapted to MLIR. Inspect post-continuation output and the selected checks before retaining a finding.

Request resolution and dialect knowledge delivery

Illustrative request shape

```
input: module.mlir
mode: specified_range
region: operation_range
checkpoint: input_profile
pipeline: continuation_profile
checking: validation_profile
objective: evaluator_profile
limits: request_budget
```

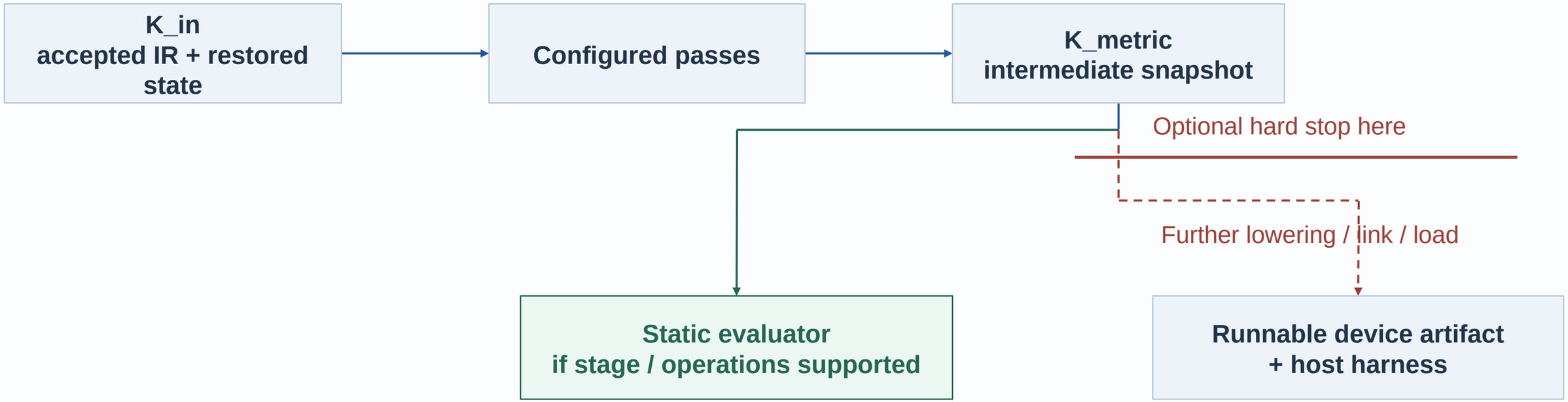
Defaults → request → per-input overrides
Shared request limits stay upper bounds.



Q09 examines public coding and specialist harness prompts. Compare direct dialect context, retrieval and added analyses.

Compilation checkpoints determine route compatibility

Illustrative continuation for both original and candidate



Runtime validation needs a runnable artifact. A hard stop before its required lowering blocks this policy. The metric checkpoint can remain earlier if continuation is allowed.

Q08 output — checkpoint catalog + validation-mode records + demonstrated / incompatible / unknown route matrix.

Validation outcomes remain independent of improvement

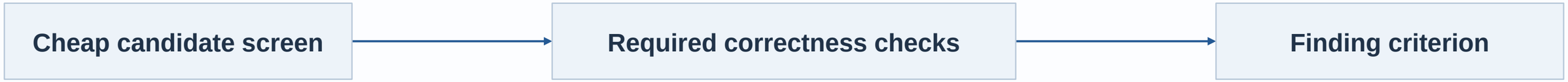
Checking route	Required material	Evidence / feasibility question
Parse, verify and compile	IR, enclosing module, target and pipeline.	Accepted structure and invariants at reached checkpoints.
Execution comparison	Runnable artifacts, inputs, harness, reference and tolerances.	Matching selected outputs on the recorded cases. Which hardware?
Semantic validation	Supported IR / operations / FP and memory model.	Equivalence or refinement within demonstrated checker coverage.

Finding eligibility

Selected checks	Metric comparison	Action
Pass	Criterion met	Retain if the replay bundle is complete.
Pass	Criterion not met	Keep attempt; return comparison feedback.
Fail or incomplete	Any result	Retain reason; repair, investigate coverage or recover setup.

Metric selection controls execution cost and interpretation

Candidate metric	Inputs / resources	Supported interpretation and open checks
Structural indicator	Chosen IR snapshot; host-side analysis.	Op counts or data-movement estimates. Check unmodeled ops and moved work.
Target cost model	Target parameters and scheduling / hardware model.	Estimated cost within model coverage. llvm-mca is a CPU reference from LPO.
Runtime measurement	Executable, workload, NPU/runtime, warm-up and repeats.	Observed execution time under the recorded conditions and aggregation.



Screening order is configurable. Finding eligibility still requires every check in the selected policy.

Feedback, stopping rules and crash recovery

Observed outcome	Feedback / next action
Compilation rejection or behavior mismatch	Send diagnostics or failing input with original/candidate identities. Agent may repair.
No qualifying improvement	Send metric values, stage, units, criterion and evidence references.
Unsupported check / unavailable hardware	Expose coverage or infrastructure cause; retain an unresolved operation.

Durable operation protocol

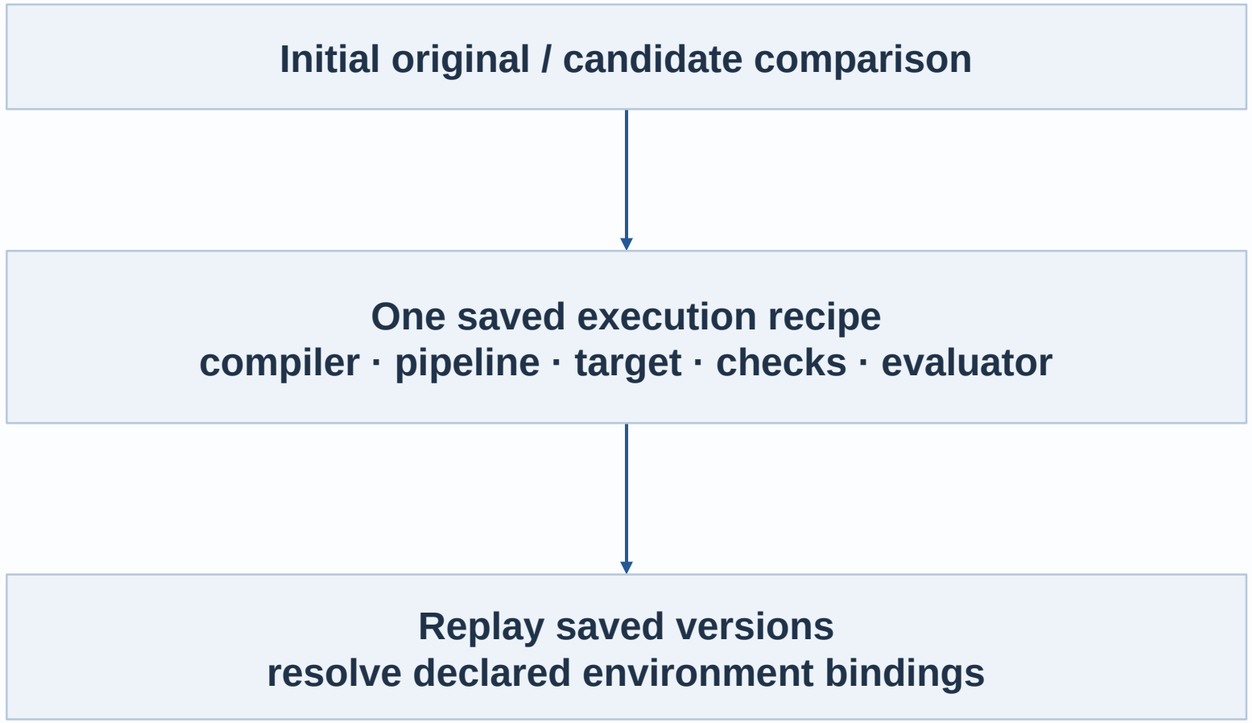


Restart → reconcile in-flight work → reuse completed stages → restore or reconstruct agent context → continue with recorded budget.

Preserve each submitted candidate and the best eligible finding independently of the current working file.

The finding is a replayable experiment

```
finding-ID/
  report.md
  manifest.json
  original/
  candidate/
  changes.diff
  evidence/
  reproduce/
  replays/
```



Proposed interface

```
framework replay ./finding-ID
```

Each replay stores fresh check results, metric samples and resource usage linked to the original finding.

Constraints resolve into an explicit experiment profile

Constraint	Resolved setting	Failure or boundary to expose
Editing scope	Whole artifact or complete operation range; context outside scope read-only.	Out-of-scope edits reject the candidate.
Semantics	Types, shapes, layouts, effects, aliasing, FP behavior and tolerances.	Additional agent assumptions need evidence.
Compilation	Start checkpoint, compiler build, pass configuration, outputs and hard stop.	Unsupported entry or incompatible continuation.
Evidence	Required checks; metric stage, units, target and improvement threshold.	Unsupported coverage and inconclusive outcomes.
Resources	Candidates, shared request limits, model calls/tokens and host/NPU jobs.	Stop scheduling beyond the remaining budget.

MVP scope and implementation stages

One engineer-supplied input mode, one demonstrated compilation route, one checking policy and one evaluator.

Stage	Deliverable
Month 1	Establish compiler entry / continuation; direct agent experiments on real cases.
Months 2–3	Implement profiles, snapshots, dialect pack, agent edits, runners, feedback and durable state.
Month 4 · MVP	Complete budget enforcement, interrupted-work resume, finding export and relocated replay.
Month 5	Evaluate held-out cases with agreed usefulness criteria, repeats and resource budgets.
Month 6	Implement and compare one context, tool, discovery or feedback refinement.

Resources — one developer, part-time compiler expertise, model/host capacity and NPU access when the selected route requires it.

Primary risks concern finding yield and integration effort

Too few useful findings

Rewrites may be invalid, trivial, narrowly applicable or already handled by a later compiler stage.

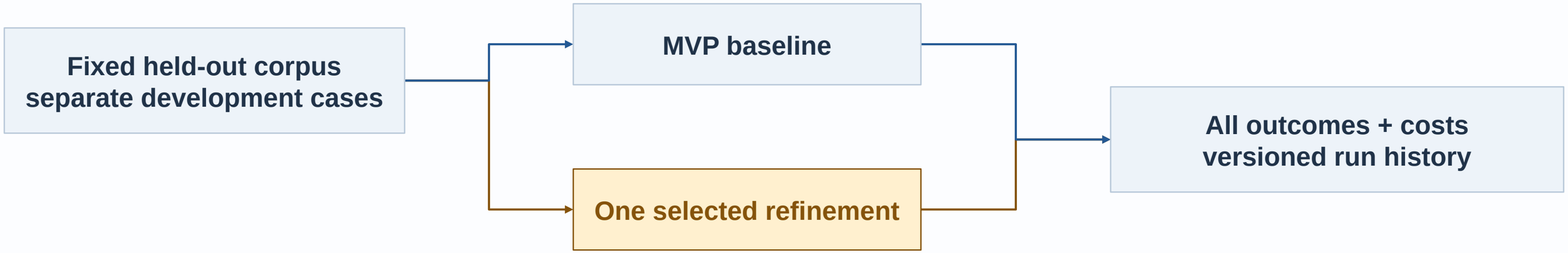
Integration costs dominate

Saved IR may require state restoration, custom harnesses, checking coverage or scarce device access.

Additional risk	Failure to watch for
Expert effort	Preparation, reviewing assumptions and maintaining the harness consume the time saved.
Metric relevance	A lower score fails to improve relevant downstream code or workloads.
Correctness coverage	Selected checks miss behavior changed by the rewrite.

Useful findings are the primary outcome. Effort saving and evidence limits are measured alongside finding quality.

Evaluation protocol and controlled refinement studies



Primary evidence	Additional outcomes
Engineer-reviewed useful findings	Rejected, duplicate, unsupported and inconclusive cases.
Replays of retained findings	Coverage, check outcomes and metric changes per replay.
At least one real case for MVP	Preparation/review effort, model use and host/NPU usage.

Before the study, agree case selection, usefulness criteria, search budgets and repeats. Any effort-saving claim identifies its reference workflow.